1.

```
global_start

_start:
        mov r1, #3
        mov r2, #5
        mov r3, #1
        mul r0, r2, r3
        add r0, r1, r0
b end
```

2.

Pipelining is a CPU design technique where multiple instruction stages are overlapped in execution — similar to an assembly line in a factory. Instead of executing one instruction at a time, the processor breaks instruction execution into stages and processes different instructions in different stages simultaneously.

Advantages of Pipelining:

- Increased throughput – more instructions completed per unit of time.
- Better hardware utilization – different CPU units work in parallel.
- Improved performance without increasing clock speed – efficiency comes from overlap, not just faster cycles.
- Scalability – deeper pipelines can handle higher frequencies (up to a point).
- Predictable instruction flow – helpful for real-time systems.

The ARM Cortex-M4 has a 3-stage pipeline:

- Fetch (F) – get the instruction from memory.
- Decode (D) – decode the instruction and read registers.

- Execute (E) – perform the operation (ALU, load/store, branch).

It has 3 stages because of:

- Balance between performance and simplicity:
  The Cortex-M4 is a microcontroller-class CPU designed for low power and deterministic execution, not maximum clock speed like high-end CPUs.
- Low latency & predictable timing:
  Fewer stages mean fewer stalls from hazards (e.g., branches or data dependencies), which is important for embedded and real-time tasks.
- Reduced power consumption:
  Shorter pipelines have less overhead in control logic and clock gating.
- Sufficient performance for target use:
  The M4 achieves high efficiency without the complexity of deeper pipelines found in desktop CPUs.

3.

The stack pointer (SP) is a special CPU register that holds the address of the top of the stack in memory. It is automatically updated when functions are called or return, when local variables are created, or when data is pushed/popped from the stack. The CPU uses SP to know where in RAM to store or retrieve temporary data.

The heap is a region of memory used for dynamic memory allocation. Unlike the stack, the heap's size can grow or shrink during program execution. Managed manually so it's more flexible but requires careful handling to avoid memory leaks or fragmentation.

In ARM Cortex-M processors, SP is stored in a dedicated CPU register called R13. When the MCU is reset, the initial value of SP is taken from address 0x00000000 in the vector table.

You should use the stack when you need small, short-lived variables whose lifetime is tied to the execution of a function, because the stack is fast, automatically managed, and requires no manual memory handling. This makes it

ideal for function parameters, return addresses, and temporary local variables. In contrast, you should use the heap when you need to store large amounts of data, data of unknown size at compile time, or data that must persist beyond the scope of the current function. The heap allows flexible, dynamic memory allocation, but it is slower and must be managed manually to avoid memory leaks or fragmentation. In short, the stack is best for quick, temporary storage with automatic cleanup, while the heap is best for long-lived or dynamically sized data requiring manual control.

4.

The number of memory locations a processor can address depends on its address bus width, not just the "bit" in its ALU.
If a CPU truly has a 128-bit address bus, then: Max memory locations = 2^{128}

If each location stores 1 byte, the total addressable memory is about 340 undecillion bytes.

Most microcontrollers use 32-bit architecture instead of 128-bit because:

- Memory needs – Most embedded systems don't require addressing more than a few gigabytes of memory; 32-bit addressing (up to 4 GB) is plenty for them.
- Cost & power – Larger architectures mean more transistors, bigger buses, and more complex circuitry, which increases cost and power consumption — not ideal for battery-powered devices.
- Performance trade-offs – Wider data paths increase chip area and may slow down some operations due to longer propagation delays.
- Software complexity – Larger registers and addresses increase code size and complexity without real benefit in most applications.
- No real hardware to match – A 128-bit address space is meaningless if the physical memory is only a few MB or GB.

5.

Instruction meaning is add the contents of R4 and R5, and store the result in R0.

Pipeline Stages

1. Fetch (F)

- The CPU fetches the binary opcode of ADD R0, R4, R5 from memory into the instruction register.
- Program Counter (PC) is incremented to point to the next instruction.

2. Decode (D)

- The instruction is decoded:
- Operation: ADD
- Source registers: R4 and R5
- Destination register: R0
- The CPU reads the contents of R4 and R5 from the register file.

3. Execute (E)

- ALU performs the addition:

  $R0 \leftarrow R4+R5$

- Result is written back into R0.

6.

Differences between RISC and CISC are:

1. Instruction Set

- RISC – Small set of simple instructions; each instruction usually executes in one clock cycle.
- CISC – Large set of complex instructions; some can execute multiple operations in a single instruction.

2. Execution Time

- RISC – Fixed, simple instructions → predictable timing and often faster per instruction.
- CISC – Variable-length, complex instructions → slower individual instruction execution.

3. Pipelining

- RISC – Easy to pipeline because all instructions are of similar size and execution time.
- CISC – Harder to pipeline efficiently due to varied instruction lengths and complexities.

4. Memory Usage

- RISC – Uses more instructions for a given task, but each instruction is small and consistent.
- CISC – Can use fewer instructions for a task because each can do more work, but instructions are larger and more complex.

5. Hardware Complexity

- RISC – Simpler hardware; more work is handled by software (compiler optimization).
- CISC – More complex hardware; decoding logic is heavier.

7.

Here's the advantages of Von Neumann architecture over Harvard architecture:

1. Simpler Design

- Uses a single memory for both instructions and data, which reduces hardware complexity compared to Harvard architecture (which needs separate memory and buses for each).

2. Lower Cost

- Fewer memory units and buses mean reduced chip size, wiring, and manufacturing costs.
- This is especially important for general-purpose computers where cost matters more than extreme speed.

3. Easier Programming

- The same memory holds both program code and data, so you can treat them uniformly.
- Self-modifying code and dynamic loading of programs are easier to implement.

4. Flexibility

- Memory can be allocated as needed between instructions and data instead of being split into fixed-size instruction and data memory blocks (as in Harvard).

5. Well-Suited for General-Purpose Systems

- Works well for systems where simplicity and cost matter more than maximum throughput — such as desktop PCs and many embedded systems.

8.

Yes — on a 32-bit processor, an instruction in memory can look exactly like data in binary form, because the CPU doesn't inherently know whether a 32-bit value is "code" or "data."

In memory, both instructions and data are just binary patterns (0s and 1s). Whether those bits are interpreted as an instruction or as data depends entirely on the context — specifically If the Program Counter (PC) points to that memory location, the CPU fetches it and decodes it as an instruction. If a load instruction (LDR, MOV, etc.) reads from that location, it treats the same bits as data.

9.

In the ARM Cortex architecture, the barrel shifter is a hardware unit connected to Operand 2 of many data-processing instructions. It allows the CPU to shift or rotate a register value on the fly as part of the instruction execution, without requiring extra instructions or cycles. This means that a single instruction can both modify a value by shifting it and use that shifted value in an arithmetic or logical operation, which improves efficiency and reduces code size.

If we consider the instruction SUBEQ a1, v7, r2, LSR #4. This is a conditional subtract instruction that executes only if the Zero flag is set. The operation subtracts a shifted version of the contents of register R2 from the contents of register R7 and stores the result in register R1. The “LSR #4” part means that before subtraction, the value in R2 is logically shifted right by 4 bits using the barrel shifter. Logical shift right fills the leftmost bits with zeros as it shifts the bits to the right.

By using the barrel shifter, the ARM processor can perform the shift and the subtraction in a single instruction cycle, instead of needing two separate instructions one to shift the operand and one to subtract. This combined operation reduces the number of instructions the processor must execute, which improves speed and saves memory. Thus, the barrel shifter plays a crucial role in making ARM instructions more compact and efficient, especially in operations involving shifted operands.

10.

To multiply a number by 50 without using any multiply instructions, we can use a combination of shifts and adds, since shifting is equivalent to multiplying by powers of two.

$50=32+16+2=2^5+2^4+2^1$

LSL R2, R0, #5

```
LSL R3, R0, #4
LSL R4, R0, #1

ADD R1, R2, R3
ADD R1, R1, R4
```

LSL is a logical shift left, which multiplies by powers of two. We add the shifted values to get the product by 50. No multiplication instructions are used.